

Week 3 Assignment Submission

Agentic Text-to-SQL System

Student: Viraj Sawad

Date: May 20, 2026

GitHub Repository:

https://github.com/VirajSawad1021/FuseAI/tree/main/Assignment_3

Assignment Overview

This assignment consists of four interconnected tasks that are completed sequentially. Each task builds upon the previous one, forming a complete **Text-to-SQL and agentic query execution system** on top of the ClassicModels PostgreSQL database developed in earlier sessions.

Task	Title	Description
1	SQL Benchmark & Evaluation Design	Ground truth SQL queries + evaluation strategy
2	Query Understanding (Decomposition)	Breaking NL questions into structured components
3	Text-to-SQL Pipeline	Automated SQL generation and execution system
4	Mini SQL Agent	Agentic system with self-correction and retry logic

Table 1: Assignment Task Summary

Database Setup

The same PostgreSQL database and dataset from previous sessions are used throughout all tasks. The database is the **ClassicModels** schema, which contains 8 tables:

- customers, orders, orderdetails
- products, productlines
- employees, offices
- payments

The database is seeded via `seed.sql` and runs on **PostgreSQL 16** inside a Docker container.

Implementation

The system is built using **FastAPI** (Python) and exposes a REST API. The full source code is organized inside the `Assignment_3/` folder of the repository.

Task 1 — SQL Benchmark & Evaluation Design

Ground truth SQL queries were written for all benchmark questions from the provided dataset. An evaluation framework was designed to assess the agent across dimensions including execution success rate, SQL correctness, join accuracy, and self-correction performance.

The evaluation strategy document is available at:

[task1_evaluation_strategy.md](#)

Task 2 — Query Understanding (Decomposition)

Each natural language question is decomposed into structured components before SQL generation:

- **Intent** — what is being asked
- **Tables** — which tables are needed
- **Columns** — which columns to retrieve
- **Filters** — WHERE conditions
- **Joins** — how tables are connected

This decomposition is performed by **GPT-4o-mini** via prompt chaining (`app/sql_generator.py`).

Task 3 — Text-to-SQL Pipeline

The pipeline follows these steps:

1. Accept a natural language question via `POST /agent/sql`
2. Decompose the question using an LLM prompt
3. Generate a PostgreSQL **SELECT** query from the decomposition
4. Validate the query (block destructive operations)
5. Execute the query against PostgreSQL
6. Return structured results

Task 4 — Mini SQL Agent (Agentic System)

The agent adds self-correction and retry logic on top of Task 3:

- On query failure, the error message is fed back to the LLM
- The agent attempts to fix and retry the query (up to **3 retries**)
- On final success, results are summarised in natural language
- All steps are logged with timestamps and execution time

Safety rule: Only **SELECT** and **WITH** queries are permitted. Any **DROP**, **DELETE**, **UPDATE**, **INSERT**, or **ALTER** is blocked by the validator.

API Endpoints

Method	Endpoint	Description
POST	/agent/sql	Main agent: NL $\rightarrow$ SQL $\rightarrow$ result $\rightarrow$ summary
GET	/customers	List customers (paginated)
GET	/customers/{id}	Get single customer
GET	/customers/count	Count of customers
GET	/overall_counts	Row counts for all 8 tables

Table 2: Key API Endpoints

Interactive API docs: <http://localhost:8000/docs> (Swagger UI)

Example Agent Response

Input:

```
POST /agent/sql
{ "question": "How many customers are from the USA?" }
```

Output:

```
{
  "sql":      "SELECT COUNT(*) FROM customers WHERE country = 'USA'",
  "result":   [{"count": 36}],
  "summary":  "There are 36 customers from the USA.",
  "status":   "success",
  "retries":  0,
  "execution_time_ms": 1842.3
}
```

Repository Link

The complete source code, evaluation strategy, task PDFs, and seed data are available at:

<https://github.com/VirajSawad1021/FuseAI/tree/main/Assignment>

The repository is **public** and contains all deliverables for Tasks 1–4.